

Logistic Regression Assignment3

Gabriel Severiche · 22 de septiembre de 2026



Contenido

Khipus.ai

Introduction to Machine Learning

Data Preparation

Classification

Logistic Regression

Assignment 3

Name: Gabriel Severiche

Instructions:

Step 1: Data Exploration

Step 2: Data Cleaning and Feature Selection

Step 3: Splitting Training and Test Data

Step 4: Build the Logistic Regression Model

Step 5: Make Predictions

Step 6: Evaluate the Model

Step 7: Visualize the Results

Plot the ROC curve

Khipus.ai

Introduction to Machine Learning

Data Preparation

© Copyright Notice 2025, Khipus.ai - All Rights Reserved.

Classification

Logistic Regression

Assignment 3

Name: Gabriel Severiche

Instructions:

1. Perform **data exploration**, **data cleaning**, **feature selection**, and **splitting training and test data** (Credit_card_Approval.csv).
2. Build the **logistic regression model**.
3. Make **predictions**.
4. Evaluate the model's performance using metrics like **accuracy**, **precision**, **recall**, and **F1 score**.
5. Visualize the results, such as plotting the **confusion matrix** and **ROC curve**

Step 1: Data Exploration

```
In [51]:  
  
# Load necessary libraries  
import pandas as pd  
  
# Load the dataset  
data = pd.read_csv('Credit_card_Approval.csv')
```

```
# Explore the data
data.head()
```

Out [51]:

	Ind_ID	GENDER	Car_Owner	Propert_Owner	CHILDREN	Annual_income	Type_Income	EDUCATION	Marital_status	Housing_type	Birthday_count	Employed_day
0	5008827	M	Y	Y	0	180000.0	Pensioner	Higher education	Married	House / apartment	-18772.0	365243
1	5009744	F	Y	N	0	315000.0	Commercial associate	Higher education	Married	House / apartment	-13557.0	-586
2	5009746	F	Y	N	0	315000.0	Commercial associate	Higher education	Married	House / apartment	NaN	-586
3	5009749	F	Y	N	0	NaN	Commercial associate	Higher education	Married	House / apartment	-13557.0	-586
4	5009752	F	Y	N	0	315000.0	Commercial associate	Higher education	Married	House / apartment	-13557.0	-586

```
In [52]:
data.info()
```

```
Out [52]:
<class 'pandas.DataFrame'>
RangeIndex: 1548 entries, 0 to 1547
Data columns (total 19 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Ind_ID                1548 non-null  int64
1   GENDER                1541 non-null  str
2   Car_Owner             1548 non-null  str
3   Propert_Owner         1548 non-null  str
4   CHILDREN              1548 non-null  int64
5   Annual_income         1525 non-null  float64
6   Type_Income           1548 non-null  str
7   EDUCATION             1548 non-null  str
8   Marital_status        1548 non-null  str
9   Housing_type          1548 non-null  str
10  Birthday_count        1526 non-null  float64
11  Employed_days         1548 non-null  int64
12  Mobile_phone          1548 non-null  int64
13  Work_Phone            1548 non-null  int64
14  Phone                 1548 non-null  int64
15  EMAIL_ID              1548 non-null  int64
16  Type_Occupation       1060 non-null  str
17  Family_Members        1548 non-null  int64
18  Approved              1548 non-null  int64
dtypes: float64(2), int64(9), str(8)
memory usage: 229.9 KB
```

```
In [53]:
data.describe()
```

Out [53]:

	Ind_ID	CHILDREN	Annual_income	Birthday_count	Employed_days	Mobile_phone	Work_Phone	Phone	EMAIL_ID	Family_Members	Approved
count	1.548000e+03	1548.000000	1.525000e+03	1526.000000	1548.000000	1548.0	1548.000000	1548.000000	1548.000000	1548.000000	1548.000000
mean	5.078920e+06	0.412791	1.913993e+05	-16040.342071	59364.689922	1.0	0.208010	0.309432	0.092377	2.161499	0.113049
std	4.171759e+04	0.776691	1.132530e+05	4229.503202	137808.062701	0.0	0.406015	0.462409	0.289651	0.947772	0.316755
min	5.008827e+06	0.000000	3.375000e+04	-24946.000000	-14887.000000	1.0	0.000000	0.000000	0.000000	1.000000	0.000000
25%	5.045070e+06	0.000000	1.215000e+05	-19553.000000	-3174.500000	1.0	0.000000	0.000000	0.000000	2.000000	0.000000
50%	5.078842e+06	0.000000	1.665000e+05	-15661.500000	-1565.000000	1.0	0.000000	0.000000	0.000000	2.000000	0.000000
75%	5.115673e+06	1.000000	2.250000e+05	-12417.000000	-431.750000	1.0	0.000000	1.000000	0.000000	3.000000	0.000000
max	5.150412e+06	14.000000	1.575000e+06	-7705.000000	365243.000000	1.0	1.000000	1.000000	1.000000	15.000000	1.000000

Step 2: Data Cleaning and Feature Selection

```
In [54]:
# Handle missing values, duplicates, and irrelevant features
missing_values = data.isnull().sum()
print(missing_values)
```

```
# Convert categorical data to numeric using one-hot encoding if needed
# Display cleaned data
```

```
Out [54]:
Ind_ID          0
GENDER          7
Car_Owner       0
Propert_Owner   0
CHILDREN        0
Annual_income   23
Type_Income     0
EDUCATION       0
Marital_status  0
Housing_type    0
Birthday_count  22
Employed_days   0
Mobile_phone    0
Work_Phone      0
Phone           0
EMAIL_ID        0
Type_Occupation 488
Family_Members  0
Approved        0
dtype: int64
```

```
In [55]:
data = data.dropna() # Remove rows with missing values
data = data.drop_duplicates() # Remove duplicates
data = pd.get_dummies(data, drop_first=True) # Convert categorical variables to dummy variables
data.head()
```

```
Out [55]:
```

	Ind_ID	CHILDREN	Annual_income	Birthday_count	Employed_days	Mobile_phone	Work_Phone	Phone	EMAIL_ID	Family_Members	...	Type_Occupation_Laborers	Typ
8	5010864	1	450000.0	-18173.0	-678	1	0	1	1	3	...	False	Fals
9	5010868	1	450000.0	-18173.0	-678	1	0	1	1	3	...	False	Fals
10	5010869	1	450000.0	-18173.0	-678	1	0	1	1	1	...	False	Fals
11	5018498	0	90000.0	-18950.0	-1002	1	1	1	0	2	...	False	Fals
13	5018503	0	90000.0	-18950.0	-1002	1	1	1	0	2	...	False	Fals

5 rows × 46 columns

```
In [56]:
missing_values = data.isnull().sum()
print(missing_values)
```

```
Out [56]:
Ind_ID          0
CHILDREN        0
Annual_income    0
Birthday_count   0
Employed_days    0
Mobile_phone     0
Work_Phone       0
Phone            0
EMAIL_ID         0
Family_Members   0
Approved         0
GENDER_M         0
Car_Owner_Y      0
Propert_Owner_Y  0
Type_Income_Pensioner  0
Type_Income_State servant  0
Type_Income_Working  0
EDUCATION_Incomplete higher  0
EDUCATION_Lower secondary  0
EDUCATION_Secondary / secondary special  0
Marital_status_Married  0
Marital_status_Separated  0
Marital_status_Single / not married  0
Marital_status_Widow  0
Housing_type_House / apartment  0
Housing_type_Municipal apartment  0
```

```
Housing_type_Office apartment      0
Housing_type_Rented apartment      0
Housing_type_With parents          0
Type_Occupation_Cleaning staff     0
Type_Occupation_Cooking staff      0
Type_Occupation_Core staff         0
Type_Occupation_Drivers             0
Type_Occupation_HR staff           0
Type_Occupation_High skill tech staff 0
Type_Occupation_IT staff           0
Type_Occupation_Laborers           0
Type_Occupation_Low-skill Laborers  0
Type_Occupation_Managers           0
Type_Occupation_Medicine staff     0
Type_Occupation_Private service staff 0
Type_Occupation_Realty agents      0
Type_Occupation_Sales staff        0
Type_Occupation_Secretaries        0
Type_Occupation_Security staff     0
Type_Occupation_Waiters/barmen staff 0
dtype: int64
```

Step 3: Splitting Training and Test Data

```
In [57]:
from sklearn.model_selection import train_test_split

# Define features and target
X = data.drop('Approved', axis=1) # Features
y = data['Approved'] # Target variable

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 4: Build the Logistic Regression Model

```
In [58]:
from sklearn.linear_model import LogisticRegression

# Initialize the model
model = LogisticRegression()
# Train the model
model.fit(X_train, y_train)
```

Out [58]:

```
▼ LogisticRegression ⓘ ?
  ► Parameters
  ► Fitted attributes
```

Step 5: Make Predictions

```
In [59]:
# Make predictions on the test set
predictions = model.predict(X_test)
```

Step 6: Evaluate the Model

Expected output:

Accuracy: 0.87 Classification Report: precision recall f1-score support

0	0.87	1.00	0.93	269
1	0.00	0.00	0.00	39
accuracy			0.87	308

macro avg 0.44 0.50 0.47 308 weighted avg 0.76 0.87 0.81 308

```
In [60]:
from sklearn.metrics import accuracy_score, classification_report

# Calculate accuracy
accuracy = accuracy_score(y_test, predictions)

# Display classification report with zero_division parameter
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:")
print(classification_report(y_test, predictions, zero_division=0))
```

```
Out [60]:
Accuracy: 0.88
Classification Report:
              precision    recall  f1-score   support

     0       0.88        1.00        0.94        180
     1       0.00        0.00        0.00         25

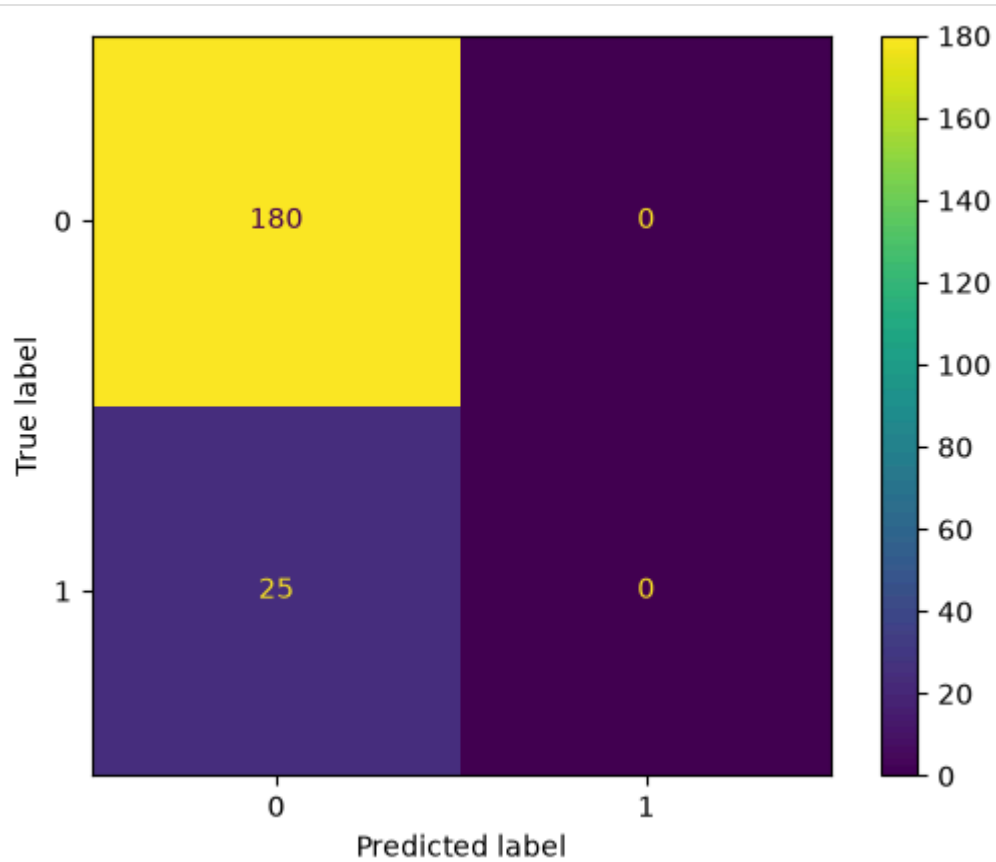
 accuracy          0.88          0.88          0.88        205
 macro avg          0.44          0.50          0.47        205
 weighted avg          0.77          0.88          0.82        205
```

Step 7: Visualize the Results

```
In [61]:
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Generate the confusion matrix
cm = confusion_matrix(y_test, predictions)
# Display the confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
```

```
Out [61]:
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1f1dceca1b0>
```



Plot the ROC curve

```
In [62]:  
import matplotlib.pyplot as plt  
from sklearn.metrics import RocCurveDisplay  
  
# Plot the ROC curve  
curva = RocCurveDisplay.from_estimator(model, X_test, y_test)  
# Plot the ROC curve for the random classifier  
curva.from_predictions(y_test, [0.5] * len(y_test), name="Random Classifier")
```

```
Out [62]:  
<sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x1f1dcf9af30>
```

